



uOttawa

L'Université canadienne
Canada's university

Digital Logic Circuits Review

Université d'Ottawa | University of Ottawa



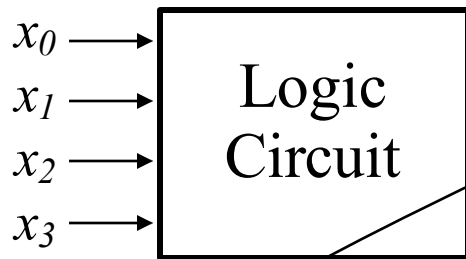
www.uOttawa.ca

Outline

- Logic Functions
- Boolean Algebra
- Logic Functions Minimization
- Logic Gates
- Combinational Circuits
- Sequential Circuits
 - Specification
 - Analysis
 - Design

Logic function

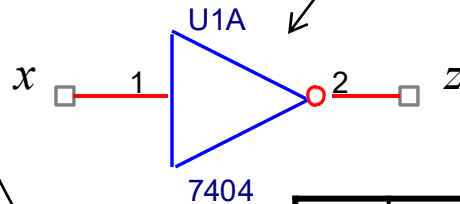
$z_i = F_i(x_0, x_1, x_2, \dots) \mid z_i \in \{0, 1\}, i = 0, 1, 2, \dots, m, x_k \in \{0, 1\}, k = 0, 1, 2, \dots, n$
 where $F_i = \text{logic functions}$; $x_k = \text{input variables}$; $z_i = \text{output variables}$



Block diagram

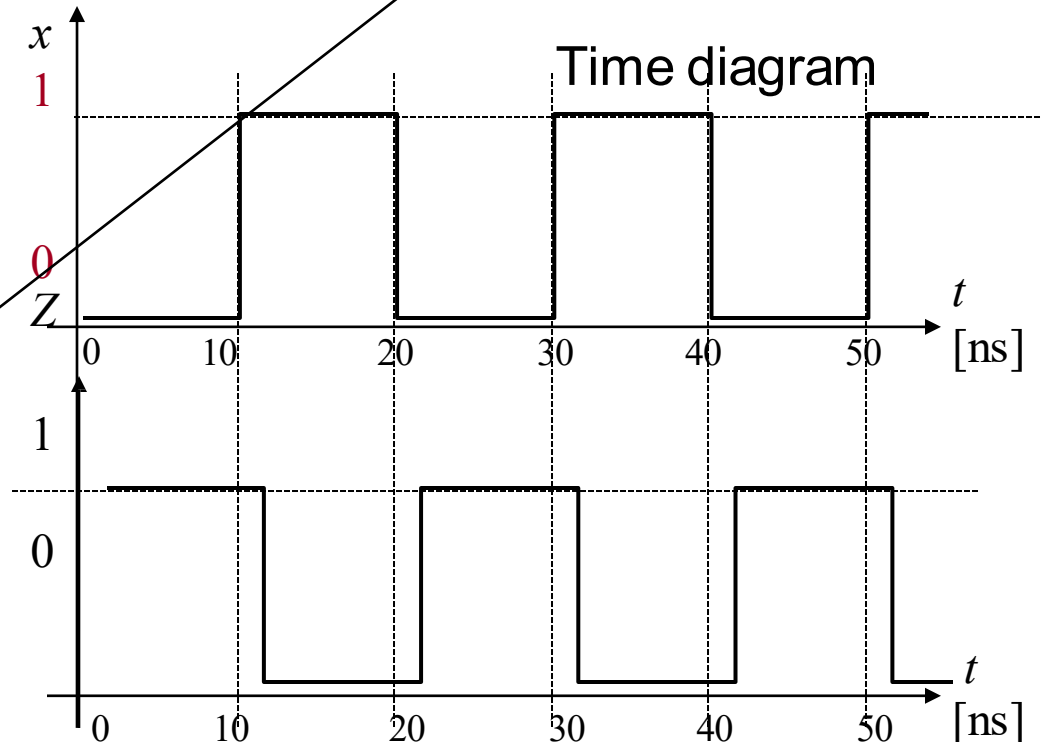
Logic functions F can be represented by logic (Boolean) expressions, truth tables, K-maps, logic diagrams of the logic circuits that implement these functions.

Logic Inverter



$$z = \overline{x}$$

x	z
0	1
1	0



BOOLEAN ALGEBRA over $\{0, 1\}$

$\{B, =, +, \bullet, \neg, 0, 1\}$, where:

B = set with at least 2 distinct elements;

$0, 1$ = 2 constants $\in B$

$+$ OR

$\bullet$ AND

$=$ equivalence relationship, with usual properties:

– reflexivity: $(\forall x \in B) \rightarrow (x = x)$

– symmetry: $(\forall x, y \in B) \rightarrow (x = y \rightarrow y = x)$

– transitivity: $(\forall x, y, z \in B) \rightarrow (x = y \text{ and } y = z \rightarrow x = z)$

Parentheses are allowed

Principle of Duality

- Every algebraic expression deducible from the postulates of Boolean algebra remain valid if the operators and identity elements are interchanged.

BOOLEAN ALGEBRA

AXIOMS

AND

$$B = \{0, 1\}$$

Closeness:

$$\forall a, b \in B$$

$$a + b \in B$$

$$a \cdot b \in B$$

Identical element:

$$\forall a \in B$$

$$a + 0 = 0 + a = a \quad [1]$$

$$a \cdot 1 = 1 \cdot a = a \quad [2]$$

Commutative:

$$\forall a, b \in B$$

$$a + b = b + a \quad [9]$$

$$a \cdot b = b \cdot a \quad [10]$$

Distributive:

$$\forall a, b, c \in B$$

$$a + (b \cdot c) = (a + b) \cdot (a + c) \quad [14]$$

$$a \cdot (b + c) = a \cdot b + a \cdot c \quad [13]$$

Complement:

$$\forall a \in B, \exists \bar{a}$$

$$a + \bar{a} = 1 \quad [7]$$

and

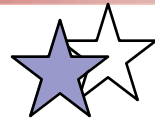
$$a \cdot \bar{a} = 0 \quad [8]$$

Associative:

$$\forall a, b, c \in B \quad (a + b) + c = a + (b + c) \quad [11]$$

$$(a \cdot b) \cdot c = a \cdot (b \cdot c) \quad [12]$$

NOTE: equation numbers, e.g. [1], are from Table 1-1 of Mano's textbook



BOOLEAN ALGEBRA

AND rules (identities)

- (2) $0 \cdot X = 0$
- (4) $1 \cdot X = X$
- (6) $X \cdot \underline{X} = X$
- (8) $X \cdot X = 0$
- (10) $X \cdot Y = Y \cdot X$
- (12) $X \cdot (Y \cdot Z) = (X \cdot Y) \cdot Z$
- (13) $\underline{X \cdot (Y + Z)} = \underline{X \cdot Y} + X \cdot Z$
- (16) $X \cdot Y = X + Y$

“ Proof ”: $X \cdot (Y + Z) = X \cdot Y + X \cdot Z$

X	Y	Z	$X \cdot (Y + Z)$	$X \cdot Y + X \cdot Z$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

NOTE: equation numbers, e.g. (2), are from Table 1-1 of Mano's textbook



BOOLEAN ALGEBRA ... continued

“ Proof ”:

OR rules (identities)

- (1) $0 + X = X$
 (3) $1 + X = 1$
 (5) $X + X = X$
 (7) $\underline{X} + \overline{X} = 1$
 (17) $\overline{(\overline{X})} = X$
- (9) $X + Y = Y + X$
 (11) $X + (Y + Z) = (X + Y) + Z$
 (14) $\underline{X} + \underline{Y} \cdot Z = (X + Y) \cdot (X + Z)$
 (15) $X + Y = \overline{\overline{X} \cdot \overline{Y}}$

X Y Z	$X + Y \cdot Z$	$(X+Y) \cdot (X+Z)$
0 0 0	0	0
0 0 1	0	0
0 1 0	0	0
0 1 1	1	1
1 0 0	1	1
1 0 1	1	1
1 1 0	1	1
1 1 1	1	1

NOTE: equation numbers, e.g. (1), are from Table 1-1 of Mano's textbook

DeMorgan's Theorems

$$\overline{\overline{X} \cdot \overline{Y}} = \overline{\overline{X}} + \overline{\overline{Y}}$$

$$\overline{\overline{X} + \overline{Y}} = \overline{\overline{X}} \cdot \overline{\overline{Y}}$$

X	Y	$\overline{\overline{X} \cdot \overline{Y}}$	$\overline{\overline{X} + \overline{Y}}$	$\overline{\overline{X}} + \overline{\overline{Y}}$	$\overline{\overline{X} \cdot \overline{Y}}$
0	0	1	1	1	1
0	1	0	0	1	1
1	0	0	0	1	1
1	1	0	0	0	0

Digital Logic Gates

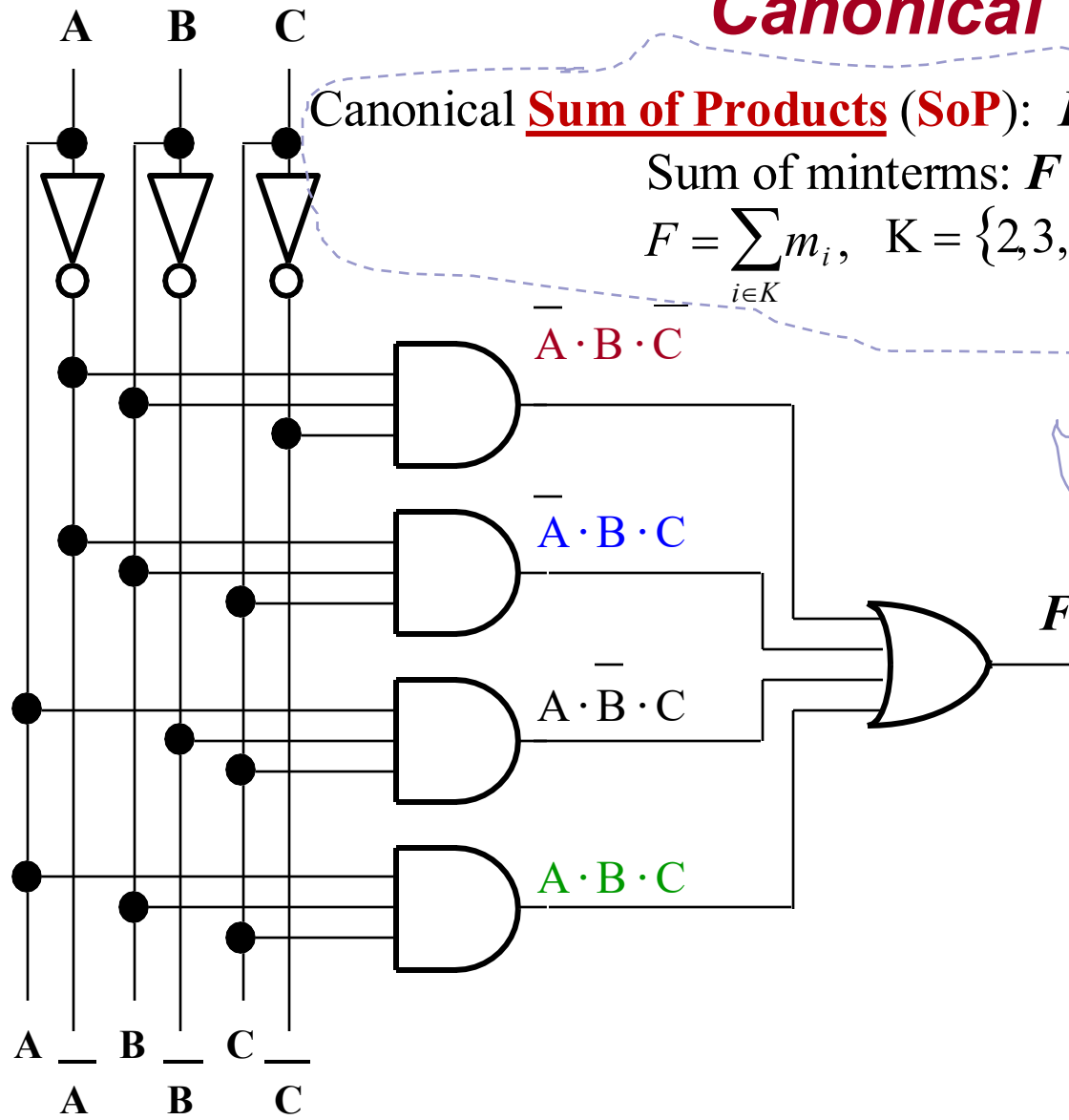
Name	Graphic symbol	Algebraic function	Truth table															
AND		$x = A \cdot B$ or $x = AB$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	x	0	0	0	0	1	0	1	0	0	1	1	1
A	B	x																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$x = A + B$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	x	0	0	0	0	1	1	1	0	1	1	1	1
A	B	x																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$x = A'$	<table><tr><th>A</th><th>x</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	x	0	1	1	0									
A	x																	
0	1																	
1	0																	
Buffer		$x = A$	<table><tr><th>A</th><th>x</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	x	0	0	1	1									
A	x																	
0	0																	
1	1																	

Name	Graphic symbol	Algebraic function	Truth table															
NAND		$x = (AB)'$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	x	0	0	1	0	1	1	1	0	1	1	1	0
A	B	x																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$x = (A + B)'$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	x	0	0	1	0	1	0	1	0	0	1	1	0
A	B	x																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$x = A \oplus B$ or $x = A'B + AB'$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	x	0	0	0	0	1	1	1	0	1	1	1	0
A	B	x																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Minterms = one product term functions

ABC	$m_0 = \bar{A}\bar{B}\bar{C}$	$m_1 = \bar{A}\bar{B}C$	$m_2 = \bar{A}B\bar{C}$	$m_3 = \bar{A}BC$	$m_4 = A\bar{B}\bar{C}$	$m_5 = A\bar{B}C$	$m_6 = AB\bar{C}$	$m_7 = ABC$
000	1	0	0	0	0	0	0	0
001	0	1	0	0	0	0	0	0
010	0	0	1	0	0	0	0	0
011	0	0	0	1	0	0	0	0
100	0	0	0	0	1	0	0	0
101	0	0	0	0	0	1	0	0
110	0	0	0	0	0	0	1	0
111	0	0	0	0	0	0	0	1

Logic functions representation using Canonical form



Canonical **Sum of Products (SoP)**: $F = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + ABC$

Sum of minterms: $F = \Sigma (m_2, m_3, m_5, m_7)$

$$F = \sum_{i \in K} m_i, \quad K = \{2, 3, 5, 7\} ; \quad F = \Sigma (2, 3, 5, 7)$$

min-term	A	B	C	F
(m ₀)	0	0	0	0
(m ₁)	0	0	1	0
(m ₂)	0	1	0	1
(m ₃)	0	1	1	1
(m ₄)	1	0	0	0
(m ₅)	1	0	1	1
(m ₆)	1	1	0	0
(m ₇)	1	1	1	1

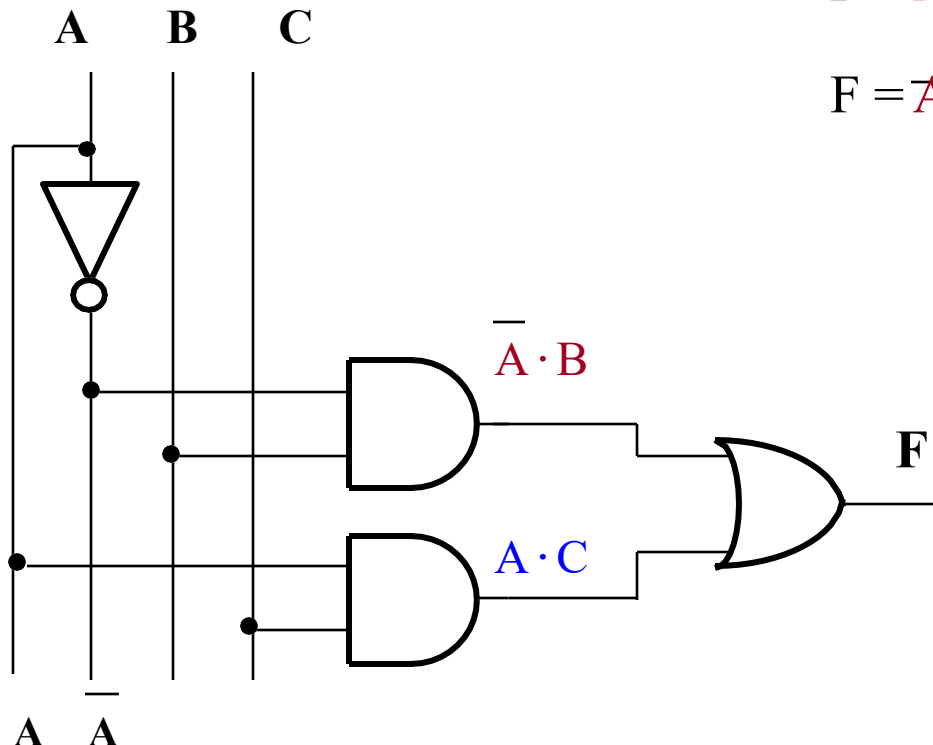
Simplifying logic functions using Boolean algebra rules

$$F = \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C + ABC$$

$$F = (\bar{A}B\bar{C} + \bar{A}BC) + (A\bar{B}C + ABC)$$

$$F = A(B\bar{C} + BC) + A(\bar{B}C + BC)$$

$$F = \underbrace{A}_{1}(\underbrace{\bar{C} + C}_{1}) + \underbrace{A}_{1}(\underbrace{\bar{B} + B}_{1})$$



$$F = \bar{A}B + AC$$



Logic functions representation using Karnaugh maps

- ★ **Karnaugh map** => graphical representation of a truth table of a logic function.
- ★ Each line in the truth table corresponds to a square in the Karnaugh map.
- ★ The Karnaugh map squares are labeled so that horizontally or vertically **adjacent squares** differ only in one variable. (*Each square in the top row is considered to be adjacent to a corresponding square in the bottom row. Each square in the left most column is considered to be adjacent to a corresponding square in the right most column.*)
- ★ Recommending for functions represented by

	A	B	C	F
(0)	0	0	0	...
(1)	0	0	1	...
(2)	0	1	0	...
(3)	0	1	1	...
(4)	1	0	0	...
(5)	1	0	1	...
(6)	1	1	0	...
(7)	1	1	1	...

sum of minterms

ABC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

logic expressions

	0	1	3	2
A	4	5	7	6

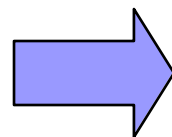
B
C

	00	01	11	10
0	0	1	3	2
1	4	5	7	6

B
C

4 variable logic functions representation on Karnaugh maps

	A	B	C	D	F
(0)	0	0	0	0	...
(1)	0	0	0	1	...
(2)	0	0	1	0	...
(3)	0	0	1	1	...
(4)	0	1	0	0	...
(5)	0	1	0	1	...
(6)	0	1	1	0	...
(7)	0	1	1	1	...
(8)	1	0	0	0	...
(9)	1	0	0	1	...
(10)	1	0	1	0	...
(11)	1	0	1	1	...
(12)	1	1	0	0	...
(13)	1	1	0	1	...
(14)	1	1	1	0	...
(15)	1	1	1	1	...



AB \ CD		C			
		00	01	11	10
00		0	1	3	2
01		4	5	7	6
11		12	13	15	14
10		8	9	11	10

D

B

Recommend for functions represented by
sum of minterms logic expressions

AB \ CD		C			
		00	01	11	10
00		0	1	3	2
01		4	5	7	6
11		12	13	15	14
10		8	9	11	10

AB \ CD		C			
		00	01	11	10
00		0	1	3	2
01		4	5	7	6
11		12	13	15	14
10		8	9	11	10

D

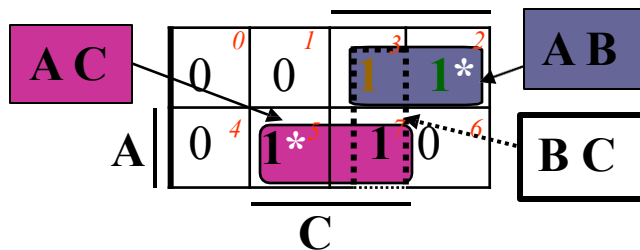
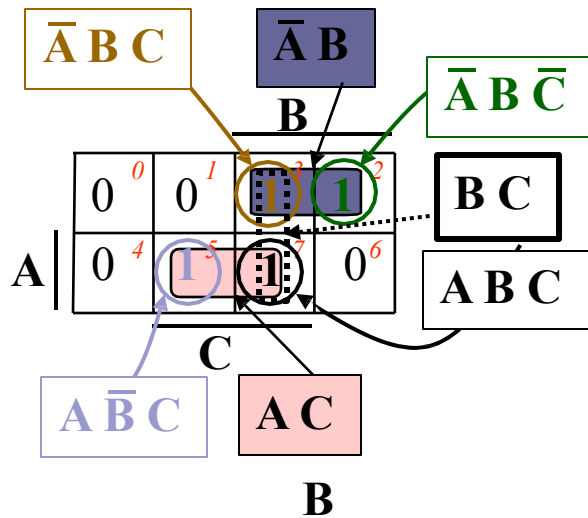
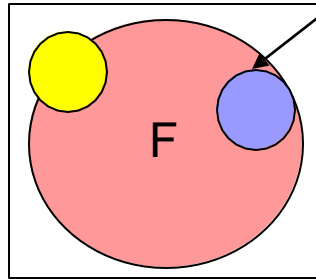
B

Implicants

Given a logic function $z = F(x_0, x_1, x_2, \dots)$

Implicant = product term that,
if equal to 1, implies $f = 1$

- whenever the implicant is 1 $\Rightarrow f = 1$
- f can be 1 other times, as well: may be implied by other implicants, than the 1 at hand



- from the K-map point of view, an implicant is a rectangle of 1, 2, 4, 8, ... (any power of 2) 1's

Prime Implicant = cannot be totally covered by another implicant
(e.g., AC , BC , $A'B$)

Each “1” which is covered by a single implicant is marked with a star (*)

Essential Prime Implicant = a prime implicant that contains at least one “1*”

A **minimum cover** of a logic function has to contain all its essential prime implicants **16**

Simplifying logic functions using Karnaugh maps

... looping

- ★ The logic function can be simplified by replacing canonical terms with a minimum cover of prime implicants, obtained by properly combining squares (**looping**) of the Karnaugh maps which contain 1s.
- ★ Looping a pair of adjacent 1s eliminates the variable that appears in both direct and complemented form.

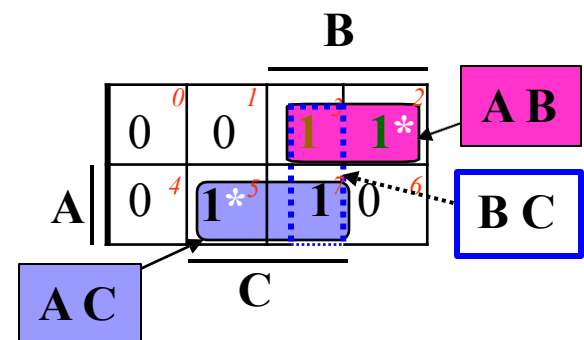
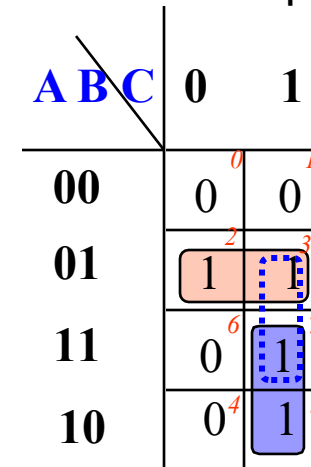
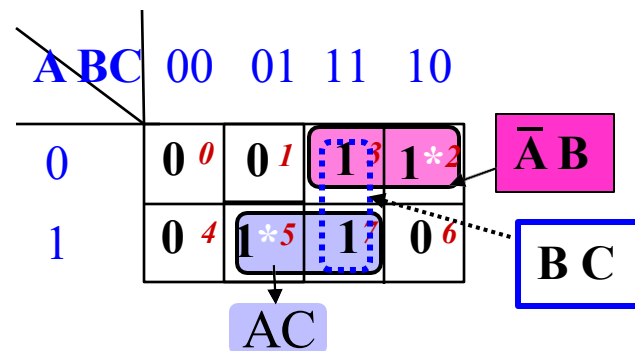
PI: $AC, BC, A'B$

EPI: $AC, A'B$

Redundant Implicant: BC

	A	B	C	F
(0)	0	0	0	0
(1)	0	0	1	0
(2)	0	1	0	1
(3)	0	1	1	1
(4)	1	0	0	0
(5)	1	0	1	1
(6)	1	1	0	0
(7)	1	1	1	1

$$F = \sum(2,3,5,7) = AC + \bar{A}B$$



Simplifying logic functions using Karnaugh maps

$$F = \sum(0,2,5,6,7,8,10,13,14,15) = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}BC\bar{D} + \bar{A}BCD + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D} + AB\bar{C}\bar{D} + ABC\bar{D} + ABCD$$

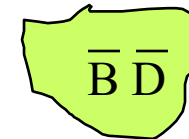
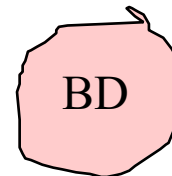
... more looping

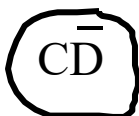
	A	B	C	D	F
(0)	0	0	0	0	1
(1)	0	0	0	1	0
(2)	0	0	1	0	1
(3)	0	0	1	1	0
(4)	0	1	0	0	0
(5)	0	1	0	1	1
(6)	0	1	1	0	1
(7)	0	1	1	1	1
(8)	1	0	0	0	1
(9)	1	0	0	1	0
(10)	1	0	1	0	1
(11)	1	0	1	1	0
(12)	1	1	0	0	0
(13)	1	1	0	1	1
(14)	1	1	1	0	1
(15)	1	1	1	1	1

★ Looping a *quad* of adjacent 1s eliminates the two variables that appears in both direct and complemented form.

CD \ AB	00	01	11	10
00	1* ⁰	0 ¹	0 ³	1 ²
01	0 ⁴	1* ⁵	1 ⁷	1 ⁶
11	0 ¹²	1* ¹³	1 ¹⁵	1 ¹⁴
10	1* ⁸	0 ⁹	0 ¹¹	1 ¹⁰

EPI*

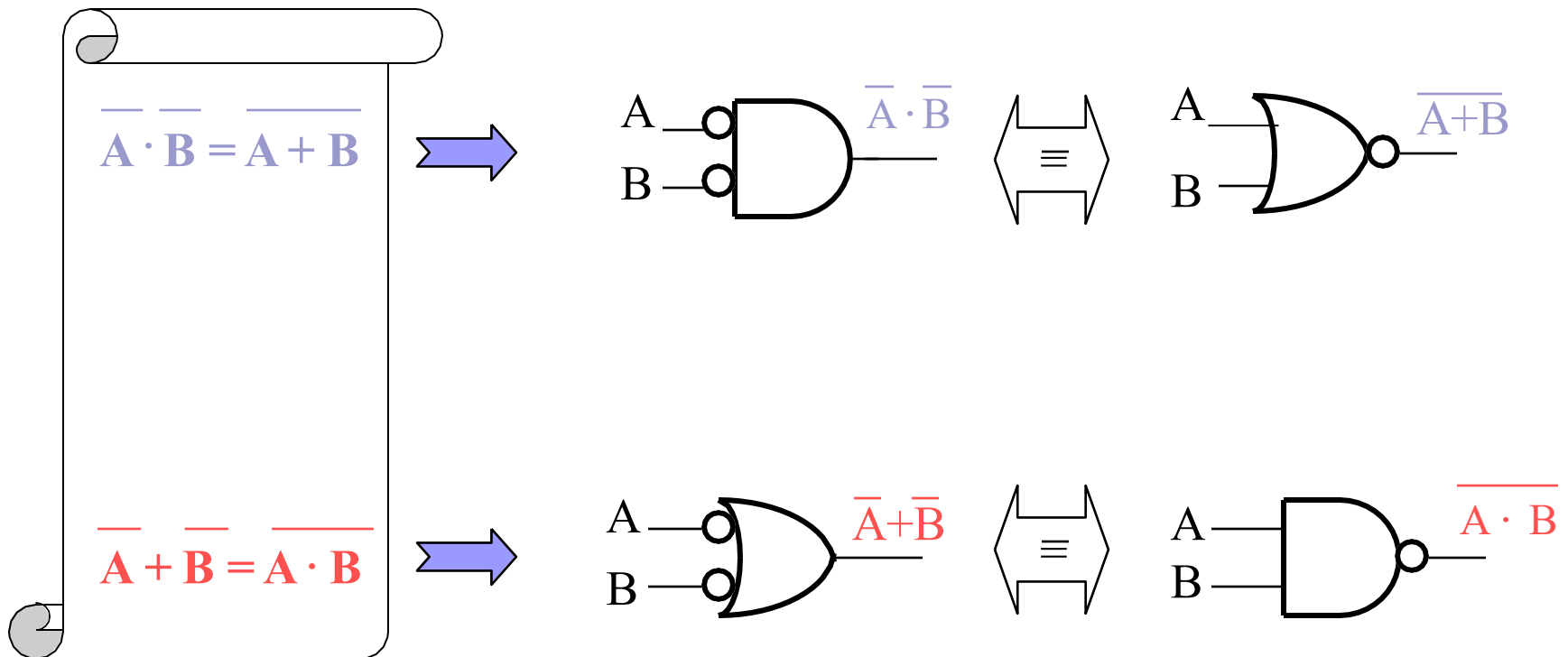

 $\bar{B}\bar{D}$

 BD
Secondary
PI

 BC

 $\bar{C}\bar{D}$

$$F = \bar{B}\bar{D} + BD + \begin{cases} \bar{C}\bar{D} \\ \text{or} \\ BC \end{cases}$$

Remember DeMorgan's Theorem

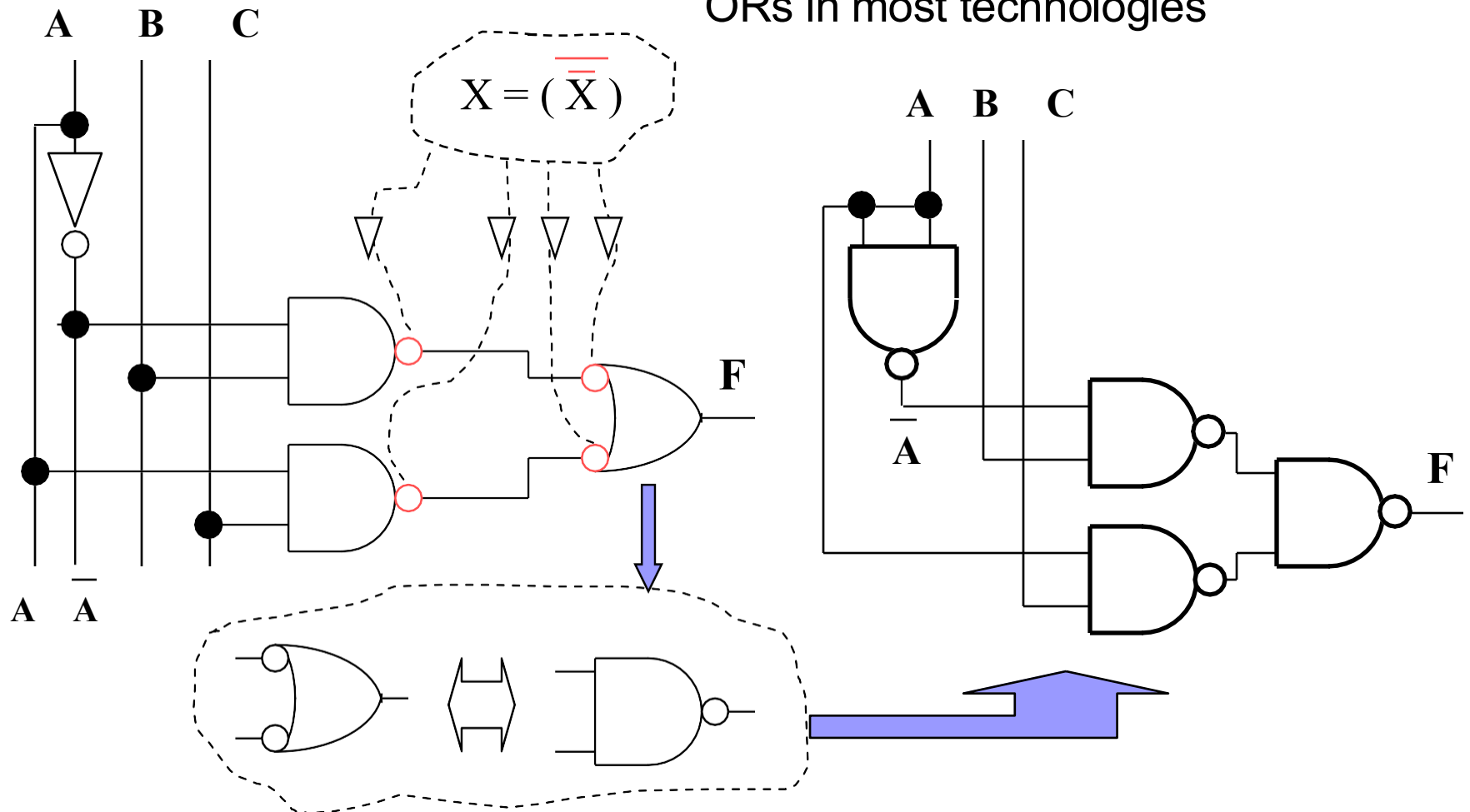
Equivalent Gate Symbols



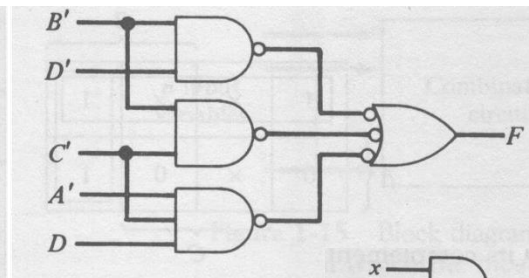
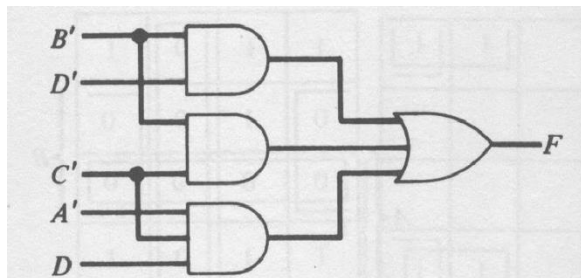
TWO-LEVEL NAND gate implementation of the “Sum-of-Product” logic functions

$$F = \bar{A}B + AC$$

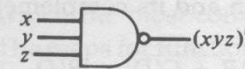
- NAND gates are faster than ANDs and ORs in most technologies



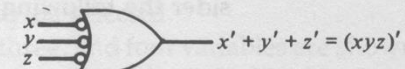
SoP & PoS implementation



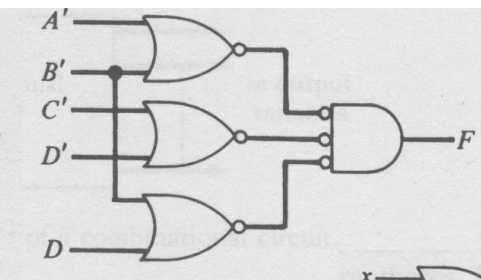
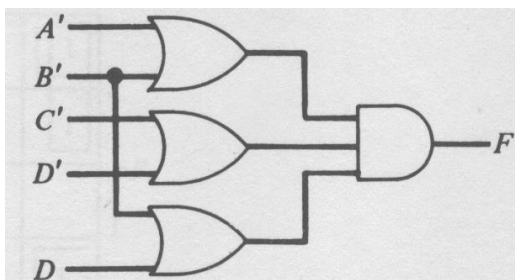
SoP = Sum of Products



(a) AND-invert



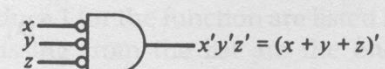
(b) invert-OR



PoS = Product of Sums



(a) OR-invert



(b) invert-AND

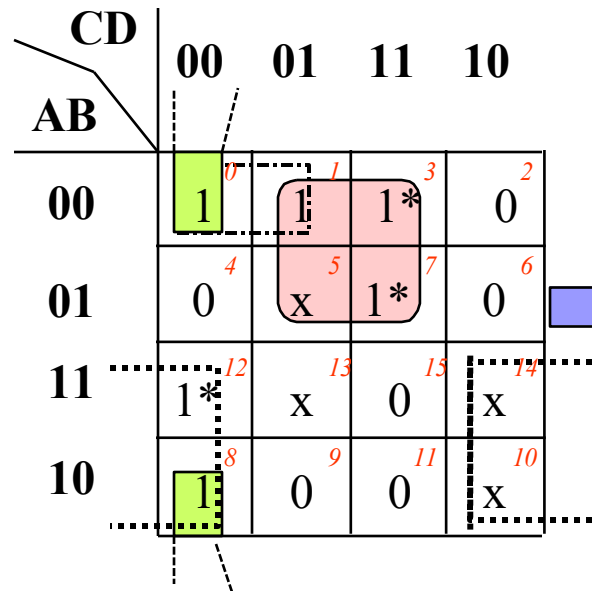
Incompletely Specified Logic Functions

$$F = \sum(0,1,3,7,8,12) + dc(5,10,13,14)$$

	A	B	C	D	F
(0)	0	0	0	0	1
(1)	0	0	0	1	1
(2)	0	0	1	0	0
(3)	0	0	1	1	1
(4)	0	1	0	0	0
(5)	0	1	0	1	x
(6)	0	1	1	0	0
(7)	0	1	1	1	1
(8)	1	0	0	0	1
(9)	1	0	0	1	0
(10)	1	0	1	0	x
(11)	1	0	1	1	0
(12)	1	1	0	0	1
(13)	1	1	0	1	x
(14)	1	1	1	0	x
(15)	1	1	1	1	0



$dc \rightarrow x = \text{don't care terms}$



EPI*

$\bar{A}\bar{D}$

Secondary
PI

$\bar{A}\bar{B}\bar{C}$

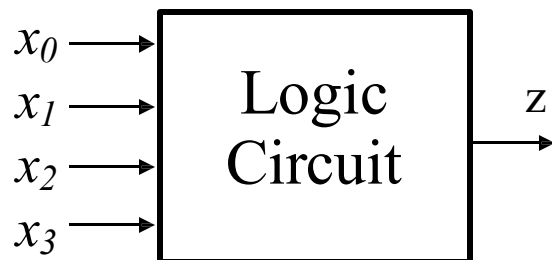
$\bar{A}D$

$\bar{B}\bar{C}\bar{D}$

$$F = \bar{A}D + \bar{A}\bar{D} + \begin{cases} \bar{A}\bar{B}\bar{C} \\ \text{or} \\ \bar{B}\bar{C}\bar{D} \end{cases}$$

Combinational Circuits

- A combinational circuit is a setup of a number of connected logic gates implementing the logic function between n input variables and m output variables.
- In a combinational circuit, the output is time-independent and does only depend on the circuit's input
- In a combinational circuit, the output is “re-computed” as soon as a change in the input occurs, and it is presented to the output with a delay



$$z = F(x_0, x_1, x_2, \dots) \mid z \in \{0, 1\}, x_k \in \{0, 1\}, k = 0, 1, 2, \dots$$

F can be represented by

- logic expression
- truth table
- K-map

Combinational Circuit Design Procedure

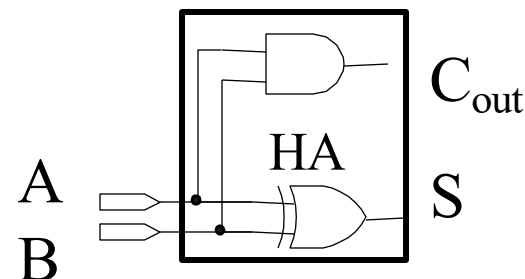
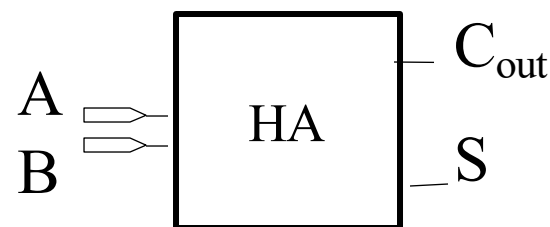
1. Define the problem
2. Assign different letter symbols to the input and output variables
3. Derive the truth table defining the relationship between the inputs and the outputs
4. Obtain the simplified boolean expression for each output variable
5. Draw the circuit's logic diagram

Half Adder

A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

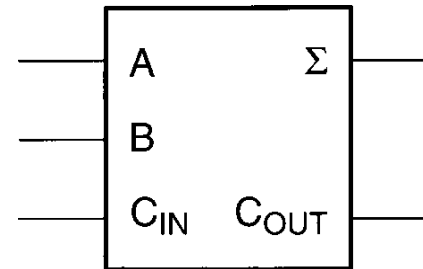
$$C_{out} = (A \cdot B)$$

$$S = A \oplus B$$



A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Full Adder



- Equations are modified as follows.
- $C_{out} = ((A \oplus B) \cdot C_{IN}) + (A \cdot B)$.
- $S = A \oplus B \oplus C_{IN}$.

